

CS-515 Advanced CS Practicum

Lab 3: Linux Socket Programming

3rd September 2026

Dr. Sreelakshmi Manjunath

Exercise 3.1

In this assignment, we will extend a simple client-server echo system, to a multiple-client, single-server echo system. The base idea is to first have a successful single client-server communication, then make the server handle multiple clients concurrently by creating a thread for each client.

The client code (`client-echo.c`) is given to you, and so is the server code (`server-echo.c`) to handle a single client. The server has a built-in delay `COMPUTE_DELAY` that simulates computation for a real non-trivial request. The default is 10 seconds which you can modify. Modify the `server-echo.c` to use `pthread`s, following the example in `pthread_hello.c`. To make things easier, we have provided a redacted code that you can edit to fill the gaps.

To test your multi-client server, open several (say 10) small terminal windows on the screen. In each, start one client and type a message **without** pressing `<Enter>`. Now switch between the windows rapidly pressing `<Enter>` in each. Since the server has a built-in delay (default is 10 seconds), it will process all the requests in parallel. If the server is working correctly, you should see the responses in each client window approximately 10 seconds after you pressed `<Enter>` in that window.

Evaluation:

You must solve this exercise with a correctly compiling server code, and a correct interaction between the client and server processes. During the demonstration, we will be asking you to invoke a few clients simultaneously, and each client will send an echo request to the server. You will be evaluated on your understanding of both the code and all aspects of the client-server interaction.

Exercise 3.2

In this assignment, we will build an FTP Client Server system. In this system, the client uploads a text file to the server. Similar to the previous assignment, the redacted code has been provided. You must edit it to complete the code.

Evaluation:

You must solve this exercise with a correctly compiling the client and server code, and a correct interaction between the client and server processes. You must demonstrate the file upload by your client application. You will be evaluated on your understanding of both the code and all aspects of the client-server interaction.

Optional

1. Instead of creating a new thread for each client, it is more efficient to execute the client requests using the **epoll** callback mechanism. Write and submit the code for

the server by modifying `server-epoll-redacted.c` to handle multiple client requests.

2. Both the `pthread`s and `epoll` mechanisms share variables between the clients. This can lead to unintended bugs and security loopholes. A safer mechanism is to **fork** a new process for each client. Modify the server to use `fork()`.
3. Extend the FTP application to multiple clients, using **pthread**, **epoll**, **fork**.

Sample Execution

Sample execution of the client, when the server is running on the local host (hence the special loopback IP address 127.0.0.1).

```
$ gcc server-echo.c -o server-echo # compile server source
$ gcc client-echo.c -o client-echo # compile client source
$ ./server-echo 5000 &             # run server, listen on port 5000
$ ./client-echo 127.0.0.1 5000     # run client, to server port
5000
```

References

1. POSIX threads tutorial, <https://hpc-tutorials.llnl.gov/posix/>
2. <https://man7.org/linux/man-pages/man7/epoll.7.html>
3. <https://www.suchprogramming.com/epoll-in-3-easy-steps/>
4. <https://jvns.ca/blog/2017/06/03/async-io-on-linux--select--poll--and-epoll/>
5. `fork()` man page, <https://www.man7.org/linux/man-pages/man2/fork.2.html>